

UNIT 1

JAVASCRIPT



Part 2 - Exercise

Client-side Web Development
2nd course – DAW
IES San Vicente 2026/2027

Índice

JavaScript Exercise 2.....3

 Part 1.....3

 Part 2.....4

JavaScript Exercise 2

This exercise is divided in two different sections where we'll work primarily with objects, classes, and collections. Download the **clinicData.js** file and add it to the folder where you'll be doing the exercise.

Part 1

Create a file called **part1.js** and import the array from file **clinicData.js**. Let's split the code into 4 different sections (display a text in the console to indicate the start of each section). It's important to note that the original array **should not be modified** at no time for any of the operations we carry out.

Section 1

Generate an iterator containing only the consultations from the patient objects. From that iterator calculate the following:

- The **total gross revenue** across all consultations.
- The **single most expensive consultation** object.

Format the total revenue using Intl.NumberFormat in Spanish (es-ES) with currency format in Euros (EUR). Example of expected output:

```
Total Revenue: 1.165,00 €  
Highest Billable Consultation: TK-2025-NEUR-77 (Neurology) - 310,00 €
```

Section 2

Define a reference Set of contraindications for a new experimental drug:

```
const drugContraindications = new Set(["penicillin", "latex", "morphine"]);
```

- Generate a collection with the names of the patients that don't have any allergy to this new drug. Use iterator methods to filter and get the names along with the isDisjointFrom Map method.

Display the names formatted with the JavaScript Internationalization API (INTL) in English. The output should look like this:

```
Name 1, Name 2, Name 3, Name 4, and Name 5
```

Section 3

Do this exercise without generating any intermediate collection:

- Filter the patients that have a valid ID using the pattern `^PAC-\d{4}-\w{2} [ !@#%*&] $`.

- We are going to anonymize those valid patients replacing some characters with regular expressions:
 - Replace all characters of the patient's name except the first letter with asterisks (e.g., "Elena Rostova" becomes "E**** R*****").
 - The expression `\B\w` refers to any alphanumeric character (`\w`) that is **not** the beginning of a word (`\B`).
 - Mask the middle part of the ID: "PAC-2022-09A#" becomes "PAC-****-09A#".
 - Return an array of formatted strings: 'Anonymized ID | Masked Name | Consultation Count'.

Section 4

- Select the consultation with ID "TK-2026-OPHT-12" from Sophia Chen.
- Schedule a follow-up appointment by adding 45 to the consultation date. If the resulting date is a weekend day (saturday or sunday), add the necessary days to set it on monday.
- Format both the original date and the adjusted follow-up date using `Intl.DateTimeFormat` with options `dateStyle: 'full'` and `timeStyle: 'short'` in Spanish (es-ES).

Print both formatted dates (last consultation and next one).

Part 2

Create the file **medical-service.class.js** with the class **MedicalService** which will have:

- Two private attributes (**serviceCode** and **baseFee**) with their corresponding getters.
 - Setter for **baseFee** ensuring it cannot be set to a value below 30; if invalid, log an error message.
- A constructor to set both values.
- Override **valueOf()** so that returning or comparing instances with mathematical or relational operators (`>`, `<`) evaluates to the `baseFee`.
- Override **toString()** returning `"[serviceCode] Base Fee: <formatted fee>"`.

Create a file **emergency-triage.class.js** (**EmergencyTriage** extends **MedicalService**) with the following characteristics:

- Private field **urgencyLevel** (number from 1 to 5). Use a setter to control this.
- Method **applyTriageCharge()** that increases the baseFee by a factor of urgencyLevel * 0.20 and returns the updated fee.
- Override toString() to include "[EMERGENCY Level <urgencyLevel>]" alongside the base service details

Create a file **lab-procedure.class.js** (LabProcedure extends MedicalService):

- Private field sampleType (string: "Blood", "Tissue", "Culture"). Control the possible values using a setter (call it from the constructor).
- Method sterilizeKit(): prints "Sterilizing testing kit for [sampleType] analysis."
- Override toString() to include "[LAB Sample: <sampleType>]".

Finally create the file **part2.js** where we will implement the functionality of this section:

Store all instantiated services in a Map<string, MedicalService> keyed by serviceCode (e.g., "SRV-01" => instance). Pre-populate the map with at least two EmergencyTriage instances and two LabProcedure instances.

We are going to execute this section with **node**, so we'll take advantage of the built-in ability to read data from the console to interact with the user. To create the object that will read user input, we'll need the following code at the beginning and end of the file:

```
import * as readline from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";

const r1 = readline.createInterface({ input, output });
// The entire program
r1.close(); // Finally we close the input/output stream
```

We'll show the user a menu until they choose the exit option (0). The menu will look something like this:

```
===== MEDICARE BILLING SYSTEM =====
1) List All Services (Formatted)
2) Register New Medical Service
3) Apply Emergency Triage Surcharges
4) Sterilize Lab Kits
5) Compare Two Services by Price (valueOf)
6) Lookup Service by Code
0) Exit
=====
```

To ask the user something and collect the answer we will use the following instruction:

```
const resp = await r1.question("Something to ask for: ");
```

Important: Do not create functions where you read from the console since readline works with promises and managing them with functions is something we have not seen yet (We will see how to use

await later).

Here's what each option will do:

1. It will only show the information (toString) of the medical services
2. Prompt for service code, base fee, and type (1 for Emergency, 2 for Lab). If Emergency, prompt for urgency level (1–5); if Lab, prompt for sample type. Store the instance in the Map
3. Iterate through all values in the Map and execute **.applyTriageCharge()** when present. Don't filter or check anything. Use the **optional concatenation operator** (?.) instead.
4. Iterate through all values in the Map and call **.sterilizeKit()**. Use the optional chaining operator **?.** Directly (same as before).
5. Prompt for two service codes. If they exist, compare the objects directly (it should use `valueOf` internally and tell the user which is cheaper and which is more expensive.
6. Prompt for a service Code. Retrieve it from the Map and print its `.toString()`, or print "Service code not registered" if it doesn't exist using `?.` and `??` together instead of `if/else`.

It's not enough that the printed results are correct. The exercise **MUST** be performed in compliance with the guidelines indicated in each section, or it may be considered invalid. At the very least, you should try to comply with them as much as possible.